

Lab 1: Requirements Elicitation & Modeling

This lab focuses on the initial steps of the Software Architecture Design process: understanding business needs and modeling the external behavior of the system using Use Case Diagrams. No complex programming is required in this foundational lab; the primary tools are document editors, modeling software, and analysis techniques.

Objectives

1. Identify and document Architecturally Significant Requirements (ASRs).
2. Identify the main Actors and Use Cases of the HRM System.
3. Model the system context using UML Use Case Diagrams.

Tool & Technology Installation

Tool	Purpose	Installation/Setup Guide
Google Docs/Microsoft Word	Documenting Requirements and ASRs.	Standard word processing software.
draw.io (Diagrams.net)	Creating professional UML Use Case Diagrams.	Access online via a web browser (no installation required) or download the desktop application.
Lucidchart / Visual Paradigm (Optional)	Alternative UML modeling tools.	Register for a free account or install the respective software.

Activity Practice 1: Requirements Elicitation

Goal: Identify and document key functional, non-functional, and architecturally significant requirements (ASRs).

Step-by-Step Instructions:

1. **Identify Actors:** Based on the HRM system scenario, list the main entities that interact with the system.

- *Action:* List at least 3 actors. (Hint: Employee (STAFF), Department Head (MANAGER), Administrator/Director (SUPERADMIN/ADMIN)).
- 2. **Document Functional Requirements (FRs):** List the specific services the system must provide to users.
 - *Action:* Document the top 5 FRs (e.g., Registration/Login via OTP, Submit leave request, Approve request, View work schedule matrix, Send/receive real-time notifications).
- 3. **Document Non-Functional Requirements (NFRs):** Define the quality attributes (how well the system performs, scales, or secures data).
 - *Action:* Document 3 critical NFRs related to Performance, Security, and Availability. (Hint: Strict RBAC implementation, Zero-latency real-time notifications, Failover tolerance when a node crashes).
- 4. **Identify Architecturally Significant Requirements (ASRs):** Choose 3 requirements (from the FRs or NFRs) that will heavily influence the architectural choice (e.g., Choosing Microservices over Monolith).
 - *Action:* For each chosen ASR, write a brief explanation of why it shapes architectural decisions.

Example ASR Documentation (ASR Card):

- **ASR:** Real-time Communication and Response.
- **Statement:** The system must immediately send push notifications (Toast Notifications) and update the number of pending requests when a new event occurs (leave submission, request approval) without requiring the client to reload the page.
- **Architectural Impact:** This requirement drives the adoption of an Event-Driven Architecture using Socket.io and Redis as a Message Broker (Pub/Sub) to synchronize states between Microservices (such as leave_service and noti_service) independently and asynchronously.

Activity Practice 2: Use Case Modeling

Goal: Visually model the system boundary and its interactions with the external environment.

Step-by-Step Instructions:

1. **Define the System Boundary:** In your chosen modeling tool (draw.io recommended), draw a large rectangle labeled "Human Resource Management System (HRM System)". This is the scope of your application.
2. **Place Actors:** Draw stick figures outside the system boundary to represent the primary Actors identified in Practice 1 (STAFF, MANAGER, SUPERADMIN).
3. **Draw Top-Level Use Cases:** Inside the system boundary, draw ovals representing the main functionalities (the FRs you identified).
 - *Employee Use Cases:* Submit leave request, Log attendance, View work schedule.
 - *Manager Use Cases:* Approve leave request, Manage department personnel, Broadcast department announcements.
 - *Director (Admin) Use Cases:* View system-wide statistics, Manage all personnel.
4. **Connect Actors to Use Cases:** Draw solid lines connecting each Actor to the Use Cases they participate in.
 - *Example:* The Employee actor connects to the "Submit leave request" Use Case. The Manager actor also connects to the "Submit leave request" Use Case (because managers can also request leave).
5. **Detail a Critical Use Case (Leave Process):** Focus on the "Submit leave request" Use Case and apply <<include>> and <<extend>> relationships.
 - *Action:* Draw the main "Submit leave request" Use Case.
 - *Use <<include>>:* Draw the "Check available leave balance" Use Case and link it to "Submit leave request" using an <<include>> arrow. This means checking the balance must obligatorily happen when submitting a request.
 - *Use <<extend>>:* Draw the "Block invalid request (Over 3 days / Shift clash)" Use Case and link it to "Submit leave request" using an <<extend>> arrow. This means the system only triggers the block/error message if the employee violates the leave limit rules.

Documentation & Submission

After completing the activities, organize your work into the following sections:

1. **ASR Documentation:** A table or list documenting the 3 chosen Architecturally Significant Requirements (ASRs) and their architectural impact (e.g., The choice of Polyglot Persistence database between PostgreSQL and MongoDB).
2. **UML Use Case Diagram:** The final diagram exported from the modeling tool, clearly showing the system boundary, actors, Use Cases, and detailed relationships (<<include>>/<<extend>>) for the Leave Request process.